

第5章 输入输出流

数据的输入与输出是应用程序的重要组成部分。程序数据的输入是从输入设备或文件将数据传送给程序中的变量或对象；程序数据的输出是将程序中对象或变量数据传送给输出设备或文件。前面主要学习了如何通过键盘和显示器等标准输入/输出（I/O）设备进行程序数据输入/输出的相关函数的使用方法。本章将通过输入/输出“流对象”概念，深入分析C++标准I/O。除了以标准输出或输入设备为对象进行输入和输出外，还经常使用磁盘文件作为输入/输出对象。磁盘文件既可以作为输入文件，也可以作为输出文件。另外，在C++中，字符串流也可作为输出/输入流对象。

5.1 C++的输入和输出流类

5.1.1 C++输入/输出的类别和特点

对于一个C++应用程序，数据输入/输出包含下列3种类别。

- (1) 系统标准设备的输入和输出（简称标准I/O）：从键盘输入数据、或输出到显示器。
- (2) 磁盘文件的输入和输出（简称文件I/O）：从磁盘文件输入或输出数据到磁盘文件。
- (3) 内存中指定的空间进行输入和输出（简称字串I/O）：通常指定一个字符数组作为存储空间（实际上可以利用该空间存储任何信息），并利用该空间进行程序数据的输入或输出。

另外，C++的输入输出操作具有如下特点。

(1) 具有类型安全性。C++输入/输出语句编译时，系统对数据类型进行严格的检查，凡是类型不正确的数据都不可能通过编译。即所谓C++的I/O操作是类型安全（type safe）的。而在C语言中，用printf和scanf进行输入输出，往往不能保证所输入输出的数据是可靠的、安全的。

(2) I/O操作的可扩展性。C++的I/O操作不仅可以用来输入/输出标准类型的数据，也可以用于用户自定义类型的数据。通过前面介绍的运算符重载机制，使得C++对标准类型的数据和对用户自定义类型数据的输入输出可以采用同样的方法处理，即C++的I/O操作具有可扩展性。

5.1.2 C++输入/输出流和流类

1. 输入/输出流的概念

C++的输入/输出“流”是指由若干字节组成的字节序列。这些字节中的数据按顺序从一个对象传送到另一对象。因此，“流”概念的外延表示信息的字节序列从源端到目的端的流动过程；而“流”概念的内涵表示在内存中为每一个数据流开辟一个内存缓冲区，流是与内存缓冲区相对应的，或者说，缓冲区中的数据就是流。在输入操作时，字节流从输入设备（如键盘、磁盘）流向内存的缓冲区。

在输出操作时, 字节流从内存的缓冲区中流向输出设备（如屏幕、打印机、磁盘等）。流中的内容可以是 ASCII 字符、二进制等形式各种编码形式的数据。

2. C++的输入/输出流类

C++系统提供了丰富的 I/O 类库。应用程序可以调用库中的不同类来实现不同的功能。在 C++ 中, 输入/输出流被定义为类。C++的 I/O 库中的类称为流类(stream class)。用流类定义的对象称为流对象。例如, cout 和 cin 就是 iostream 类中构建的全局对象。因此, 使用 cout 和 cin 进行操作不是 C++语言中提供的语句, 而是调用全局对象来实现。

(1) iostream 类库中常用类

C++编译系统提供了用于输入/输出的 iostream 类库。该类库中包含许多用于输入输出的类。表 5-1 归纳了一些 C++常用的 I/O 类。其继承层次见图 5-1 所示。

类名	作用	在哪个头文件中声明
ios	抽象基类	iostream
istream	通用输入流和其他输入流的基类	iostream
ostream	通用输出流和其他输出流的基类	iostream
iostream	通用输入输出流和其他输入输出流的基类	iostream
ifstream	输入文件流类	fstream
ofstream	输出文件流类	fstream
fstream	输入输出文件流类	fstream
istrstream	输入字符串流类	strstream
ostrstream	输出字符串流类	strstream
strstream	输入输出字符串流类	strstream

表 5-1 I/O 库中的常见流类

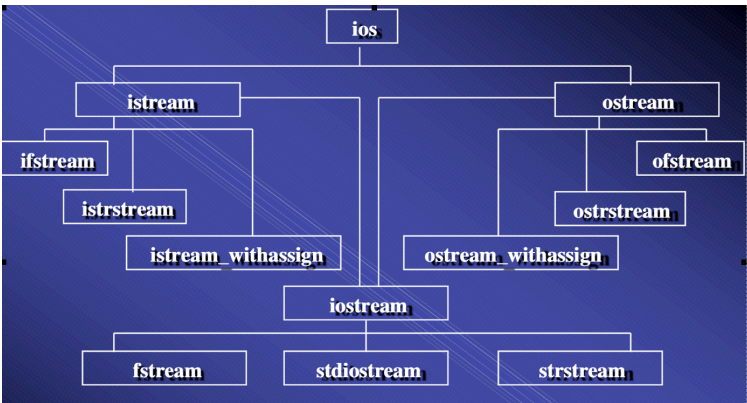


图 5-1 ios 类簇的继承层次关系

其中: ios 是抽象基类, 直接派生出 istream 类和 ostream 类。iostream 类是从 istream 类和 ostream 类通过多重继承而派生的类。C++对文件的输入输出需要用 ifstream 和 ofstream 类。ifstream 支持对文件的输入操作; ofstream 支持对文件的输出操作。类 ifstream 继承了类 istream;

类 `ofstream` 继承了类 `ostream`；类 `fstream` 继承了类 `iostream`。另外，I/O 类库中还有后面将要介绍字符串流类 `istrstream` 和 `ostrstream` 等。

(2) `iostream` 类库相关的头文件

头文件是应用程序与标准类库的接口。`iostream` 类库中不同类的声明被放在不同的头文件中。或者说，`iostream` 类库的接口是分别采用不同的头文件来实现。C++常用的 I/O 操作的头文件有：

`iostream` 头文件，包含对输入输出流进行操作所需的基本信息。

`fstream` 头文件，用于用户管理的文件的 I/O 操作。

`strstream` 头文件，用于字符串流 I/O 操作。

`stdiostream` 头文件，用于混合使用 C 和 C++的 I/O 操作机制。

`iomanip` 头文件，使用格式化 I/O 操作。

注意，上述头文件都没有带扩展名（*.h）。这是新版的标准化 C++头文件写法。带扩展名（例如 `iostream.h`）是 VC6.0 及以前老版本的写法。

(3) `iostream` 头文件中定义的流对象

在上述诸多头文件中，最常见的是 `iostream` 头文件，它包含了对输入输出流进行操作所需的基本信息。因此，大多数 C++编译系统都包括 `iostream`。在 `iostream` 头文件中不仅申明了相关的 I/O 类，还定义了多种全局的流对象，例如，常用流对象有：`cin`：标准输入流对象，键盘为其对应的标准设备。`cout`：标准输出流对象，显示器为标准设备。`cerr` 和 `clog`：标准错误输出流，输出设备是显示器。`cerr` 为非缓冲区流；`clog` 为缓冲区流，一旦错误发生立即显示。

(4) `iostream` 头文件中的运算符重载

“<<” 和 “>>” 运算符在 C++中基本操作为“左位移”运算符和“右位移”运算符。但在 `iostream` 头文件中对它们进行了重载，使得这 2 个运算符能用作标准类型数据的输入和输出操作。因此，只要在应用程序中用 `#include “iostream”` 语句，就可以直接使用运算符的重载功能来直接对标准类型的数据进行 I/O 操作。例如，采用 `ostream operator << (int )` 语句就可以向输出流插入一个 `int` 数据。因此，在 `iostream` 类中已将运算符 “>>” 和 “<<” 重载为对以下标准类型的提取运算符：`char`, `signed char`, `unsigned char`, `short`, `unsigned short`, `int`, `unsigned int`, `long`, `unsigned long`, `float`, `double`, `long double`, `char*`, `signed char*`, `unsigned char*` 等。另外 “<<” 运算符除了以上的标准类型外，还增加了一个 `void*` 类型。

如果想将 “<<” 和 “>>” 用于用户自定义类型数据的 I/O 操作，就不能简单地采用包含 `iostream` 头文件来解决，必须由程序设计者采用前面介绍运算符重载方法来对 “<<” 和 “>>” 进行重载。这一点在后面章节将详细介绍。

5.2 标准的输出流/输入流

前面介绍了 C++的输出、输入基本概念和特点。下面就具体分析 C++的标准输出流和输入流的相关类和对象。

5.2.1 标准输出流

C++在 `iostream` 头文件中不仅定义了标准输出流类以及输出流对象，还定义了需要输出操作函数。另外，还在 `iomanip` 头文件中定义了输出操作控制格式和标识符等。下面就这些内容进行详细介绍。

1. 标准输出流对象

`ostream` 类定义了 3 个输出流对象，分别是：

(1) `cout` 流对象。`cout` 是 `console output` 的缩写。`cout` 是 `ostream` 流类在 `iostream` 文件中构建的全局对象。使用 “`cout<<`” 输出基本类型的数据时，系统会判断输出数据的类型，选择调用与之匹配的运算符重载函数进行输出操作。`cout` 流对象在内存中对应开辟了一个缓冲区，用来存放流中的数据。

(2) `cerr` 流对象。标准错误流，已被指定与显示器关联。`cerr` 的作用是向标准错误设备 (standard error device) 输出有关出错信息。`cerr` 与标准输出流 `cout` 用法根本不同之处：`cout` 流通常是传送到显示器输出，但也可以被重定向输出到磁盘文件；而 `cerr` 流中的信息只能在显示器输出，它不能被重定向。一般在调试程序时，往往不希望程序运行时的出错信息被送到其他文件，只是在显示器上及时输出，这时用 `cerr` 比 `cout` 更适合。另外，`cerr` 流中的信息是用户根据需要指定的。

(3) `clog` 流对象。`clog` 流对象也是标准错误流，它是 `console log` 的缩写。作用是在终端显示器上显示出错信息，这一点和 `cerr` 相同。差别是：`cerr` 是不经过缓冲区，直接向显示器上输出有关信息，而 `clog` 中的信息先存放在缓冲区中，然后输出。

下面通过一个程序实例来说明 `cout` 流对象和 `cerr` 流对象的使用方法及其差别。

实例 5-1

```
#include <iostream>
#include <fstream>
using namespace std;

void TestWide()
{
    int i = 0;
    cout << "Enter a number:";
    cin >> i;
    cerr << "test2 for cerr" << endl;
    clog << "test2 for clog" << endl;
}

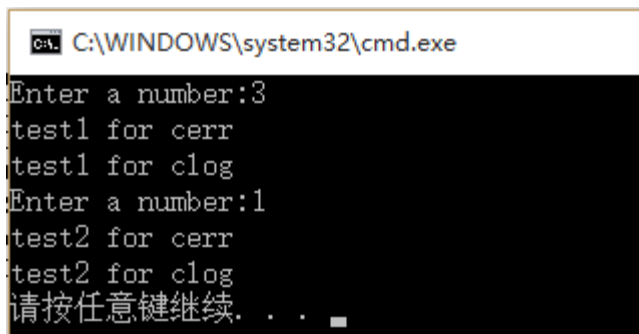
int main()
{
    int i = 0;
    cout << "Enter a number:";
```

```

cin >> i;
cerr << "test1 for cerr" << endl;
clog << "test1 for clog" << endl;
TestWide();
}

```

运行结果:



提问：如果将上述程序运行结果定向到磁盘文件，显示器输出结果有什么不同？

2. 格式输出

在输出数据时,有时希望程序数据按指定的格式输出。有两种方法可以达到此目的。一种方法是使用控制符的方法;另一种方法是使用流对象的有关成员函数。

(1) 使用控制符控制输出格式

在头文件 `iomanip` 中定了不同的格式控制符（如表 5-2 所示），使用控制符就可以实现按照指定的格式输出数据。因此，在程序中如果使用这些格式控制符，就需要包含 `iomanip`。下面通过程序实例来说明这些格式控制符使用。

表5-2 控制符控制输出/输入格式

控制符	作用
<code>dec</code>	设置整数的基数为10
<code>hex</code>	设置整数的基数为16
<code>oct</code>	设置整数的基数为8
<code>setbase(n)</code>	设置整数的基数为n(n只能是16, 10, 8之一)
<code>setfill(c)</code>	设置填充字符c, c可以是字符常量或字符变量
<code>setprecision(n)</code>	设置实数的精度为n位。在以一般十进制小数形式输出时, n代表有效数字。在以fixed(固定小数位数)形式和scientific(指数)形式输出时, n为小数位数。
<code>setw(n)</code>	设置字段宽度为n位。
<code>setiosflags(ios::fixed)</code>	设置浮点数以固定的小数位数显示。
<code>setiosflags(ios::scientific)</code>	设置浮点数以科学计数法(即指数形式)显示。
<code>setiosflags(ios::left)</code>	输出数据左对齐。

setiosflags(ios::right)	输出数据右对齐。
setiosflags(ios::skipws)	忽略前导的空格。
setiosflags(ios::uppercase)	在以科学计数法输出E和十六进制输出字母X时，以大写表示。
setiosflags(ios::showpos)	输出正数时，给出“+”号。
resetiosflags	终止已设置的输出格式状态，在括号中应指定内容。

对表中的控制符的解释如下：

- (1) setw(int n) 预设输出宽度
如：cout<<setw(6)<<123<<endl;
输出结果为“ 123”，在123的前面会有3个空格，123右对齐。
- (2) setfill(char c) 预设填充字符。
如：cout<<setfill(' #')<<123<<endl;
输出显示结果为“###123”，123右对齐，在前面填充3个'#’。
- (3) setbase(int n) 预设整数输出进制。
如：cout<<setbase(8)<<255<<endl;
输出显示结果为377
- (4) setprecision(int n) 用于控制输出流显示浮点数的精度，整数n代表显示的浮点数数字的个数。但需要特别注意：setprecision(n)和setiosflags(ios::scientific)结合使用时，setprecision(n)表示浮点数在用指数形式输出时小数位数。在和setiosflags(ios::fixed)结合使用时，setprecision(n)表示在固定的小数形式输出时小数的位数（VS2012编译器）。

下面通过实例程序来说明使用方法。

实例5-2 通过控制符控制输出精度格式。

```
#include <iostream>
#include <iomanip> //格式控制
using namespace std;

void main()
{
    double amount = 22.0 / 7;
    cout << amount << endl; // (1) 以默认格式输出
    cout << setprecision(0) << amount << endl; // (2) 设置有效位数为0等于默认格式
    cout << setprecision(1) << amount << endl; // (3) 设置有效位数为1
    cout << setprecision(2) << amount << endl; // (4) 设置有效位数为2
    cout << setprecision(3) << amount << endl; // (5) 设置有效位数为3
    cout << setprecision(4) << amount << endl; // (6) 设置有效位数为4
    cout << setiosflags(ios::fixed);
    cout << setprecision(8) << amount << endl; // (7) 设置有效位数为8

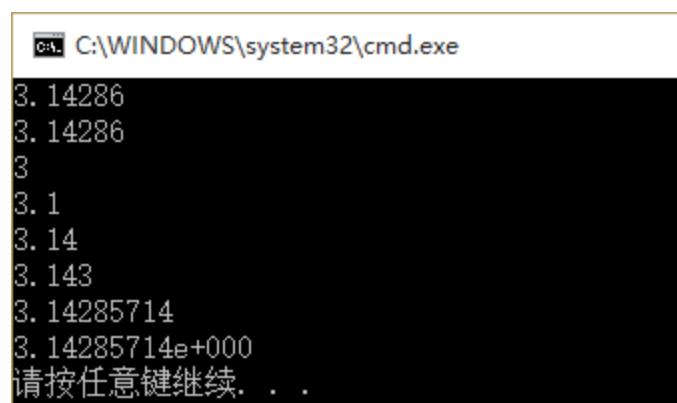
    cout << resetiosflags(ios::fixed); // 由于fixed与scientific不互斥，同时设置会输出
    // 出错，所以这里取消fixed设置
}
```

```

cout << setiosflags(ios::scientific) << amount << endl; // (8) 设置以科学计数法显示
cout << setprecision(6);
}

```

运行结果：



第1行输出数值之前没有设置有效位数，所以用流的有效位数默认设置值6：第2个输出设置了有效位数0，C++最小的有效位数为1，所以使用默认设置为6来处理：第3~6行输出按设置的有效位数输出。第7行输出是与setiosflags(ios::fixed)合用。所以setprecision(8)设置的是小数点后面的位数，而非全部数字个数。第8行输出用setiosflags(ios::scientific)来表示指数表示的输出形式。其有效位数沿用上次的设置值8。

（2）通过流对象的成员函数控制输出格式

在流对象 cout 中定义了多种控制格式输出的成员函数（如表 5-3 所示）。通过调用成员函数也可以控制程序数据的输出格式。其中，成员函数 setf 和控制符 setiosflags 括号中的参数表示格式状态，它是通过格式标志来指定的。格式标志在类 ios 中被定义为枚举值。因此在引用这些格式标志时要在前面加上类名 ios 和域运算符“::”。格式标志见书中如表 5-4 所示。

表 5-3 流对象的控制输出格式成员函数

流成员函数	与之作用相同的控制符	作用
precision(n)	setprecision(n)	设置实数的精度为n位
width(n)	setw(n)	设置字段宽度为n位
fill(c)	setfill(c)	设置填充字符c
setf()	setiosflags()	设置输出格式状态, 括号中应给出格式状态，内容与控制符setiosflags括号中内容相同
unsetf()	resetiosflags()	终止已设置的输出格式状态

表5-4. 设置格式状态的格式标志

格式标志	作用
ios::left	输出数据在本域宽范围内左对齐
ios::right	输出数据在本域宽范围内右对齐

ios::internal	数值的符号位在域宽内左对齐，数值右对齐，中间由填充字符填充
ios::dec	设置整数的基数为10
ios::oct	设置整数的基数为8
ios::hex	设置整数的基数为16
ios::showbase	强制输出整数的基数(八进制以0打头，十六进制以0x打头)
ios::showpoint	强制输出浮点数的小点和尾数0
ios::uppercase	在以科学计数法输出E和十六进制输出字母X时，以大写表示
ios::showpos	输出正数时，给出“+”号
ios::scientific	设置浮点数以科学计数法(即指数形式)显示
ios::fixed	设置浮点数以固定的小数位数显示
ios::unitbuf	每次输出后刷新所有流
ios::stdio	每次输出后清

需要特别指出的是，这些格式标志在ios是一个公共的枚举类型(enum)。当格式标志被全部清零时，输出流将采用默认格式显示。而当多个互斥的格式标志被同时置为1时，输出流将按照预设的优先级选择格式进行输出。

下面通过程序实例来说明这些格式控制符使用方法。

实例5-3 通过流对象cout的控制成员函数输出数据。

```
#include <iostream>
using namespace std;
int main()
{
    int a = 666;
    cout.setf(ios::showbase);           //显示基数符号(0x或0)
    cout << "dec:" << a << endl;        //默认以十进制形式输出a
    cout.unsetf(ios::dec);               //终止十进制的格式设置
    cout.setf(ios::hex);                 //设置以十六进制输出的状态
    cout << "hex:" << a << endl;        //以十六进制形式输出a
    cout.unsetf(ios::hex);               //终止十六进制的格式设置
    char *pt = "beijing";
    cout.width(8);                       //指定域宽为8
    cout << pt << endl;                 //输出字符串
    cout.width(8);
    cout.fill('#');                      //指定空白处以'#'填充
    cout << pt << endl;
    double pi = 22.0 / 7.0;              //输出pi值
    cout.setf(ios::scientific);          //指定用科学记数法输出
    cout << "pi=";
    cout.width(13);                       //指定域宽为13
    cout << pi << endl;
    cout.unsetf(ios::scientific);        //终止科学记数法状态
```



```

cout.setf(ios::fixed);           //指定用定点形式输出
cout.width(13);                  //指定域宽为13
cout.setf(ios::showpos);        //正数输出“+”号
cout.setf(ios::internal);       //数符出现在左侧
cout.precision(8);              //保留位小数
cout << pi << endl;
return 0;
}

```

运行情况如下：



```

C:\WINDOWS\system32\cmd.exe
dec:666
hex:0x29a
 beijing
#beijing
pi=3.142857e+000
+##3.14285714
请按任意键继续. . .

```

dec:666	(十进制形式)
hex:0x29a	(十六进制形式, 以 x 开头)
beijing	(域宽为 8)
#beijing	(域宽为 8, 空白处以 '#' 填充)
pi=3.142857e+000	(指数形式输出, 域宽 13, 默认位小数)
+##3.14285714	(小数形式输出, 精度为 8, 最左侧输出数符 "+")

从上面的实例可以看出，对输出格式的控制，既可以用控制符（实例5-2），也可以用cout流的有关成员函数(如实例5-3)，二者的作用是相同的。控制符是在头文件*iomanip*中定义的，使用时必须包含*iomanip*头文件。cout流的成员函数是在头文件*iostream*中定义的，使用时只需包含头文件*iostream*，不必包含*iomanip*。另外，在使用格式控制符和流函数时，还有几点需要特别注意：

1) 成员函数width(n)和控制符setw(n)只对其后的第一个输出项有效。如果要求在输出数据时都按指定的同一域宽n输出，不能只调用一次width(n)，而必须在输出每一项前都调用一次width(n)。

2) 在表5-2中的输出格式状态是分组的，每组中同时只能选用一种(例如，dec，hex和oct中只能选一)。在用成员函数setf和控制符setiosflags设置输出格式状态后，如果想改设置为同组的另一状态，应当调用成员函数unsetf(对应于成员函数setf)或resetiosflags(对应于控制符sefiosflags)，先终止原来设置的状态，再设置其他状态。

3. 其他输出成员函数。

iostream类除了提供上面介绍过的用于格式控制的成员函数外，还提供了成员函数put()。该函数的功能是输出单个字符。在程序中一般用cout和插入运算符“<<”实现输出，cout流在内存中有相应的缓冲区。有时用户还有特殊的输出要求，例如只输出一个字符。cout.put('a');调用该函

数的结果是在屏幕上显示一个字符a。put函数的参数可以是字符或字符的ASCII代码（也可以是一个整型表达式）。例如

```
cout.put(65 + 32); 也显示字符a，因为97是字符a的ASCII代码。
```

可以在一个语句中连续调用put函数。

```
如：cout.put(71).put(79).put(79).put(68).put('\n');
```

在屏幕上显示GOOD。

5.2.2 标准输入流

标准输入流是从标准输入设备(键盘)流向程序的数据。C++在头文件 iostream 中定义了输入全局对象 cin。另外还定义了一系列流对象 cin 中的成员函数。下面分别介绍。

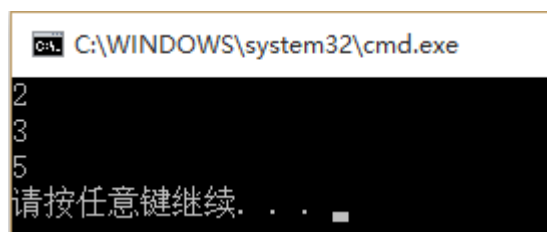
1. 输入流对象 cin

cin 是 istream 类中构建的全局对象。该对象是通过流提取符“>>”从标准输入设备(键盘)读取数据。然后保存在程序中的变量或对象中。流提取符“>>”从输入流中读取数据时，一般跳过输入流中的空格、tab 键、换行符等符号。另外，只有在输入完数据再按回车键后，该行数据才被送入缓冲区，形成输入流，提取运算符“>>”才能从中提取数据。下面通过程序实例来说明输入流对象 cin 的使用方法。

实例 5-4 cin 基本使用方法介绍

(1) 用法1：最基本，也是最常用的用法，输入一个数字：

```
#include <iostream>
using namespace std;
int main()
{
    int a, b;
    cin >> a >> b;    //用法1: 输入数字
    cout << a + b << endl;
    return 0;
}
```



输入：2[回车]3[回车]

输出：5

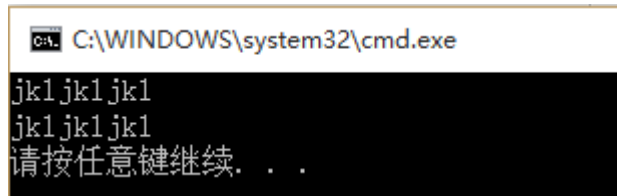
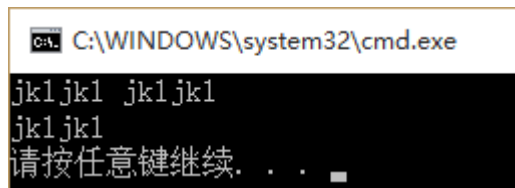
(2) 用法2：接受一个字符串，遇“空格”、“TAB”、“回车”都结束。

```
#include <iostream>
using namespace std;
int main()
```

```

{
    char a[20];
    cin >> a;           //用法2: 输入一个字符串
    cout << a << endl;
    return 0;
}

```

输入: jkljkljkl

输出: jkljkljkl

输入: jkljkl jkljkl //遇空格结束

输出: jkljkl。

2. istream 类流对象中成员函数

除了可以用 cin 输入标准类型的数据外, 还可以用 istream 类的流对象中的一些成员函数, 实现字符的输入。

(1) cin.get() 函数

该函数读入一个字符后, 流成员函数 get 有 2 种形式: 分别是无参数形式和有参数形式, 3 种使用方法。

1) 不带参数的 get() 函数的调用形式为 cin.get()。

其功能是用来从指定的输入流中提取一个字符, 函数的返回值就是读入的字符。若遇到输入流中的文件结束符, 则函数值返回文件结束标志 EOF (End Of File)。

2) 有一个参数的 get() 函数调用形式为 cin.get(ch)

其作用是从输入流中读取一个字符, 赋给字符变量 ch。如果读取成功则函数返回非值 (真), 如失败 (遇文件结束符) 则函数返回值 (假)。

3) 有多个参数的 get() 函数调用形式为 cin.get(字符数组, 字符个数 n, 终止字符)

其作用是从输入流中读取 n-1 个字符, 赋给指定的字符数组 (或字符指针指向的数组), 如果在读取 n-1 个字符之前遇到指定的终止字符, 则提前结束读取。如果读取成功则函数返回非值 (真), 如失败 (遇文件结束符) 则函数返回值 (假)。

下面通过程序实例来说明这 3 种形式的使用方法。

实例 5-5 用 get() 函数读入字符。

```

#include <iostream>
using namespace std;

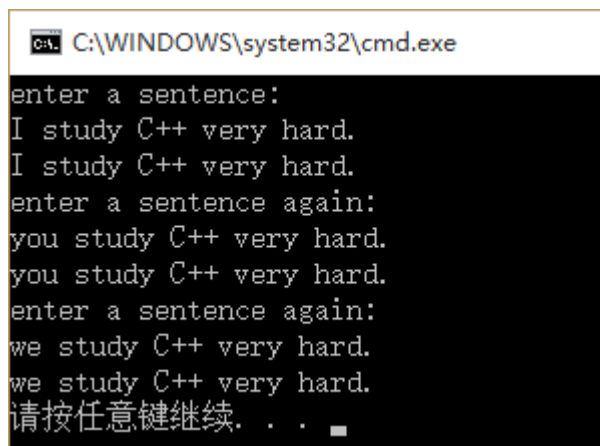
```

```

int main()
{
    char c;
    char ch[30];
    cout << "enter a sentence:" << endl;
    //读取一个字符赋给变量c, 如果成功, 返回字符ASCII码值
    while ((c = cin.get()) != '\n')
        cout.put(c);
    cout << "\nenter a sentence again:" << endl;
    //读取一个字符赋给变量c, 如果读取成功, cin.get(c)为真
    while (cin.get(c) && c != '\n')
        cout << c;
    cout << "\nenter a sentence again:" << endl;
    cin.get(ch, 30, '\n'); //指定换行符为终止字符
    {cout << ch << endl; }
    return 0;
}

```

运行情况如下:



```

C:\WINDOWS\system32\cmd.exe
enter a sentence:
I study C++ very hard.
I study C++ very hard.
enter a sentence again:
you study C++ very hard.
you study C++ very hard.
enter a sentence again:
we study C++ very hard.
we study C++ very hard.
请按任意键继续. . .

```

```

enter a sentence:
I study C++ very hard.  (输入一行字符)
I study C++ very hard.  (输出该行字符)
enter a sentence again:
you study C++ very hard (输入一行字符)
you study C++ very hard
enter a sentence again:
we study C++ very hard  (输入一行字符)
we study C++ very hard  (输出该行字符)
^Z (程序结束)

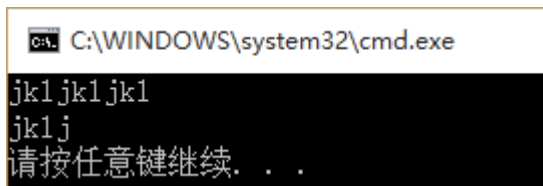
```

(2) cin.getline(char * a, int n, char c)函数

getline()函数的作用是从输入流中读取一行字符。其用法与带3个参数的get()函数类似。即cin.getline(字符数组(或字符指针),字符个数n,终止标志字符)。例如,如果下列程序设计接受一个字符串,可以接收空格并输出,需包含“#include<string>”。下面通过程序来说明其使用方法。

实例5-6

```
#include <iostream>
using namespace std;
int main(void)
{
    char m[20];
    cin.getline(m, 5); //当第三个参数省略时,系统默认为'\0'
    cout << m << endl;
    return 0;
}
```

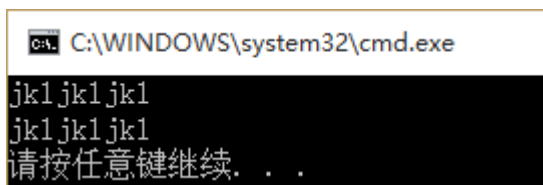


输入: jkljkljkl

输出: jklj

接受5个字符到m中,其中最后一个为'\0',所以只看到4个字符输出:

如果把5改成20:



输入: jkljkljkl

输出: jkljkljkl

输入: jklf fjlsjf fjsdklf

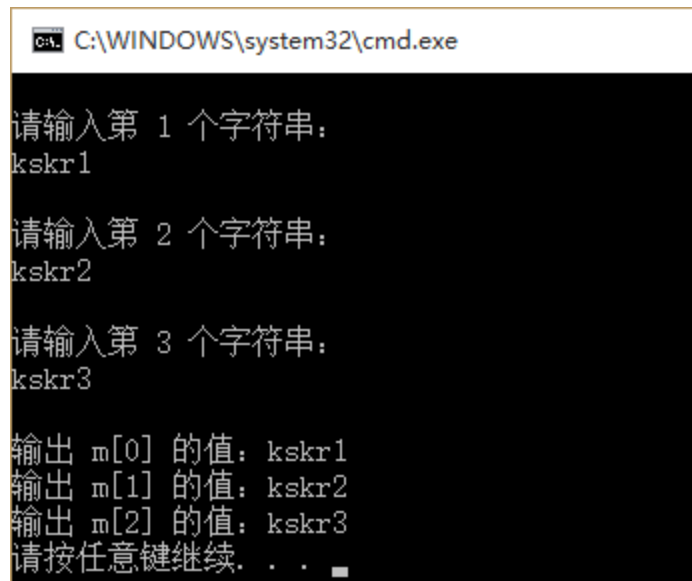
输出: jklf fjlsjf fjsdklf

如果将例子中cin.getline()改为cin.getline(m,5,'a');当输入jkljkljkl时输出jklj,输入jkaljkljkl时,输出jk。当用在多维数组中的时候,也可以用cin.getline(m[i],20)之类的用法。例如:

实例5-7

```
#include <iostream>
#include <string>
using namespace std;
int main(void)
{
    char m[3][20];
    for (int i = 0; i < 3; i++)
    {
        cout << "\n请输入第 " << i + 1 << " 个字符串: " << endl;
        cin.getline(m[i], 20);
    }
    cout << endl;

    for (int j = 0; j < 3; j++)
        cout << "输出 m[" << j << "] 的值: " << m[j] << endl;
    return 0;
}
```



```
C:\WINDOWS\system32\cmd.exe

请输入第 1 个字符串:
kskr1

请输入第 2 个字符串:
kskr2

请输入第 3 个字符串:
kskr3

输出 m[0] 的值: kskr1
输出 m[1] 的值: kskr2
输出 m[2] 的值: kskr3
请按任意键继续. . .
```

```
请输入第 1 个字符串:
kskr1
请输入第 2 个字符串:
kskr2
请输入第 3 个字符串:
kskr3
输出 m[0] 的值: kskr1
输出 m[1] 的值: kskr2
输出 m[2] 的值: kskr3
```

(3) `Cin.ignore(int n, char c)` 函数

`Cin.ignore()` 是从输入流(`cin`)中提取字符, 提取的字符被忽略(`ignore`), 不被使用。每抛弃一个字符, 它都要计数和比较字符。如果计数值达到 `n` 或者被抛弃的字符 `c`, 则 `cin.ignore()` 函数执行终止; 否则, 它继续等待, 如 `ignore(5, 'A')` 表示跳过输入流中 5 个字符, 遇 'A' 后就不再跳了。它的一个常用功能就是用来清除以回车结束的输入缓冲区的内容, 消除上一次输入对下一次输入的影响。比如可以这么用: `cin.ignore(1024, '\n')`; 通常把第一个参数设置得足够大, 这样实际上总是只有第二个参数 '\n' 起作用, 所以这一句就是把回车(包括回车)之前的所有字符从输入缓冲(流)中清除出去。

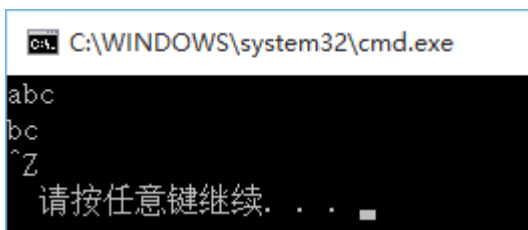
另外, `ignore()` 函数也可以不带参数或只带一个参数。如 `ignore()` 表示 `n` 默认值为 1, 终止字符默认为 EOF。即相当于 `ignore(1, EOF)`

(4) `cin.peek()` 函数

`peek()` 函数的作用是观测下一个字符。其调用形式为 `c=cin.peek()`。`cin.peek()` 函数的返回值是指针指向的当前字符, 但它只是观测, 指针仍停留在当前位置, 并不后移。如要访问字符是文件结束符, 则函数值是 EOF(-1)。下面通过程序实例来说明该函数的使用方法。

实例5-8

```
#include <iostream>
using namespace std;
int main(void)
{
    char ch, temp;
    while (cin.get(ch)) {
        temp = cin.peek();
        cout.put(temp);
    }
    return 0;
}
```



假如输入 `abc`,

则输出 `bc`。

该程序设计思想大概是: 先用 `cin.get(ch)` 把 `abc` 插入流中, 当前流位置在 `a` 处。通过 `temp = cin.peek()` 把当前流的下一字符的副本 `b` 返回给 `temp`, 所以输出 `b`。然后通过循环, 当前流位置在 `b` 处, 再通过 `temp = cin.peek()` 返回流的下一字符 `c` 给 `temp`, 所以输出 `c`。

5.2 文件 I/O 操作与文件流

迄今为止，程序中的数据输入、输出都是以系统指定的标准设备为对象的。但在实际应用中，常以磁盘文件作为对象。即从磁盘文件读取数据，将数据输出到磁盘文件。本节将重点介绍文件流的输入和输出操作。

5.2.1 文件类型和文件流

所谓“文件”，一般指存储在外部介质上数据的有序集合。文件是操作系统对数据进行管理的组织单位。按照内容来看，文件一般可以分为程序文件(program file)和数据文件(data file)。按照数据编码格式，数据文件可以进一步分为：ASCII 文件和二进制文件。

所谓 ASCII 数据文件是指数据是以字符的 ASCII 码格式组织在文件中。二进制文件是指数据按照内存中不同类型的数据格式直接组织在文件中。对于字符类型数据，在内存中是以 ASCII 代码形式存储的。因此，无论用 ASCII 文件还是用二进制文件都是一样的。但是对于其他类型数据，使用 ASCII 文件还是用二进制文件，二者存在很大差异。例如，长整数使用补码格式，在内存中占个 8 字节。如果使用二进制文件，是直接使用其在内存中的编码格式，在文件中也占个 8 字节。如果将它转换为 ASCII 码文件存在，则需要将补码编码格式转换 ASCII 码编码格式，则要占个 14 字节，见图. 5-2。

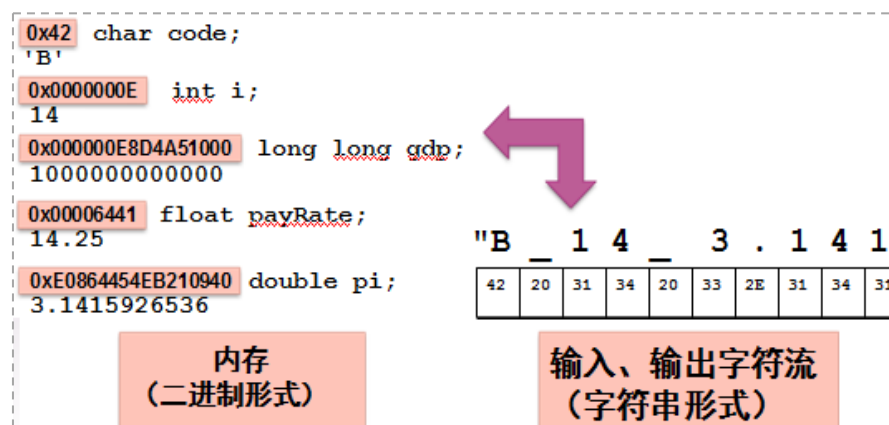


图5-2 ASCII文件和二进制文件的差别

在 C++ 中还有一个重要的概念是“文件流”。所谓“文件流”是以磁盘文件为输入输出对象，它们与应用程序中的对象（变量）之间进行数据传递的数据流。根据数据流向，可以分为输出文件流和输入文件流。其中，输出文件流是从程序对象流向磁盘文件对象的数据流，输入文件流是从磁盘文件对象流向应用程序对象的数据流。每个文件流都有一个内存缓冲区与之对应。

5.2.2 文件流类与文件流对象

在 C++ 的 I/O 类库中定义了 3 种文件类。这些类也包含在 `iostream` 头文件中。用于文件操作的文件类具体包括如下 3 种。

- (1) `ifstream` 类。从 `istream` 类派生的，用来支持从磁盘文件的输入。
- (2) `ofstream` 类。从 `ostream` 类派生的，用来支持向磁盘文件的输出。
- (3) `fstream` 类。从 `iostream` 类派生的，用来支持对磁盘文件的输入和输出。

前面介绍过用标准设备为对象的输入和输出时，需要定义流对象。例如，`cin` 和 `cout` 等。而且 `cin` 和 `cout` 已有编译系统软件在 `iostream` 中构建好了，用户可以直接使用。类似，如果要以磁盘文件为对象进行输入和输出，也必须构建文件流类的对象。而且与标准设备的输入和输出不同，这些文件流类的对象（简称文件流对象）必须由用户自己来构建。那么如何来建立文件流对象呢？下面就介绍文件流对象的打开和关闭等操作。

(1) 文件打开操作

在 C++ 中，文件操作一般是经历 3 个步骤：打开文件，读写文件和关闭文件。所谓打开文件就是构建文件流对象的过程。该过程包含 2 方面的含义：（1）为文件流对象和指定的磁盘文件之间建立关联，以便使得文件流能流向指定文件。（2）指定文件的工作方式。

C++ 提供了 2 种不同的方法实现文件打开操作。

方法 1：调用文件流的成员函数 `open()` 来打开文件流对象。其调用的一般格式如下。

文件流对象.`open`(磁盘文件名, 输入输出方式)。

例如：`ofstream outfile; //定义 ofstream 类(输出文件流类)对象 outfile`

`outfile.open("file1.dat", ios::out); //使文件流与 file1.dat 文件建立关联`

磁盘文件名可以包括路径。如 `"c:\\c++\\file1.dat"`。如缺省路径，则默认为当前目录下的文件。

方法 2：在构建文件流对象时指定参数，调用文件流类的有参构造函数来实现打开文件的功能。

例如：

`ostream outfile("fl.dat", ios::out);`

其中，在文件打开时，输入输出方式是在 `ios` 类中定义的，它们是枚举常量，有多种选择，见下面表 5-5。

表 5-5 文件工作方式

方式标志	解释
<code>ios::app</code>	追加写。以输出方式打开文件，并输出数据写道文件尾
<code>ios::ate</code>	打开已有文件，打开后文件指针指向文件尾。 <code>ios::app</code> 就包含有此属性
<code>ios::binary</code>	以二进制方式打开文件。缺省方式是文本方式打开
<code>ios::in</code>	文件以输入方式打开
<code>ios::out</code>	文件以输出方式打开（缺省方式）。打开时

<code>ios::nocreate</code>	不建立文件，所以文件不存在时，则打开失败
<code>ios::noreplace</code>	不覆盖文件，所以文件存在时，则操作失败
<code>ios::trunc</code>	打开文件，如果文件存在，把文件长度设为0，删除文件已有内容。如果不存在，则建立新文件；如果指定了 <code>ios::out</code> ，而没有指定 <code>ios::app</code> 和 <code>ios::ate</code> ，则默认是该方式

需要特别注意 4 点。

①有些新版本的 I/O 类库中不提供 `ios::nocreate` 和 `ios::noreplace`。

②每个打开的文件都有一个文件数据当前位置标记，标记文件当前读写位置。

③可以用“位或”运算符“|”对输入输出方式进行组合。例如：`ios::app|ios::nocreate`

④如果打开操作失败，使用 `open()` 成员函数的返回值为(假)，使用构造函数方式（方式 2）则流对象的值为 0。也可以用成员函数 `fail()` 来测试打开成功与否等状态信息。

⑤对于 `ifstream`，默认的打开模式是 `ios::in`。`ifstream` 只能用于输入，它没有提供任何用于输出的操作。打开 `ifstream` 文件流时，采用 `ios::out`、`ios::app`、`ios::trunc`、`ios::ate` 等这些与输出有关的打开模式是毫无意义的。同理，对于 `ofstream`，默认的打开模式是 `ios::out`。`ofstream` 只能用于输出，它没有提供任何用于输入的操作。打开 `ofstream` 文件流时，采用 `ios::in` 这样与输入有关的模式是毫无意义的。另外，对于 `fstream`，没有默认的打开模式，因此在打开文件时必须在 `ios::in`、`ios::out`、`ios::in|ios::out` 这三个打开模式中指定一个。

(2) 关闭文件

在对已打开文件的读写操作完成后，应关闭该文件。关闭文件用成员函数 `close()`。其调用一般格式如下。

文件流对象.`close()`;

例如：`outfile.close()`;

关闭文件主要含义是解除该文件与文件流对象之间的关联，并刷新对应文件流的缓冲区。

5.2.3 ASCII 文件的读写操作

ASCII 文件中的数据是以字节为单位，均以 ASCII 编码方式来组织。即一个字节存放一个字符。所谓，ASCII 文件的读写操作是以字节（字符）为单位来进行读写，应用程序可以从 ASCII 文件中读入若干个字符，也可以向它输出一些字符。

ASCII 文件的读写操作有两种方法。

(1) 用流插入运算符“<<”和流提取运算符“>>”输入输出标准类型的数据。从 ASCII 文件插入和提取标准类型数据时，数据类型及其编码格式的转换是由系统自动完成。

(2) 使用文件流对象的成员函数进行字符的输入输出。在文件流类中，包含成员函数有 `put()`、`get()`、`getline()` 等。

下面结合程序实例来说明上述 2 种方法的使用。

例5-9 将百鸡问题计算结果存入文件。

```

#include <iostream>
#include <iomanip>
#include <fstream>
using namespace std;

int main()
{
    int i, j, k;
    ofstream ofile; //构建文件流对象
    ofile.open("d:\\myfile.txt"); //打开输出文件
    ofile << "    公鸡        母鸡        小鸡" << endl; //标题写入文件
    for (i = 0; i <= 20; i++)
        for (j = 0; j <= 33; j++)
        {
            k = 100 - i - j;
            if ((5 * i + 3 * j + k / 3 == 100) && (k % 3 == 0)) //注意(k%3==0)非常重要
                ofile<<setw(6)<<i<< setw(10) << j << setw(10) << k << endl; //数据写入文件
        }
    ofile.close(); //关闭文件
    return 0;
}

```

例5-10 读出存放百鸡问题计算结果的文件（见例5-9）。

```

#include<fstream>
#include<iostream>
#include<iomanip>
using namespace std;

int main()
{
    char a[28];
    ifstream ifile("d:\\myfile.txt"); //作为输入文件打开
    int i = 0, j, k;

    while (ifile.get(a[i])) //读标题,请对比cin.get(),不可用>>,它不能读白字符
    {
        if (a[i] == '\n') break;
        i++;
    }

    a[i] = '\0';
    cout << a << endl;
    while (1)
    {

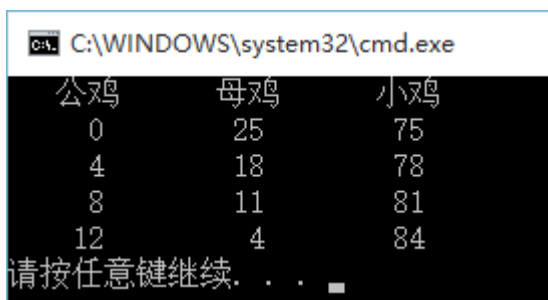
```

```

    ifile >> i >> j >> k; //由文件读入数据
    if (ifile.eof() != 0) break; //当读到文件结束时，ifile.eof() 为真
    cout << setw(6) << i << setw(10) << j << setw(10) << k << endl; //屏幕显示
}

ifile.close(); //关闭文件
return 0;
}

```



```

C:\WINDOWS\system32\cmd.exe
公鸡    母鸡    小鸡
  0      25     75
  4      18     78
  8      11     81
 12       4     84
请按任意键继续. . .

```

公鸡	母鸡	小鸡
0	25	75
4	18	78
8	11	81
12	4	84

说明：

- (1) 对比文件打开的2种方式使用差异。
- (2) 打开文件时，如磁盘文件不存在，会自动建立文件，但指定目录必须存在，否则建立文件失败。
- (3) 在程序前增加一句： `#include<fstream>`。为什么？不要是否可以？
- (4) 在向磁盘ASCII文件输出一个数据后，要输出一个(或几个)空格或换行符，以作为数据之间的分隔。否则，以后从磁盘文件读数据时，存入的数字连成一片无法区分。

5.2.4 二进制文件的读写操作

二进制文件是将内存中数据存储形式不加转换地传送到磁盘文件，因此，二进制文件读写操作速度比 ASCII 文件快。二进制文件打开时，要用 `ios::binary` 指定为以二进制形式传送和存储。另外，二进制文件除了可以作为输入文件或输出文件外，还可作为输入/输出的文件。

- (1) `read()` 和 `write()` 成员函数

对二进制文件的读写主要用 `iostream` 类的成员函数 `read()` 和 `write()` 来实现。函数的原型如下所示。

```

istream& read(char *buffer,int len);
ostream& write(const char * buffer,int len);

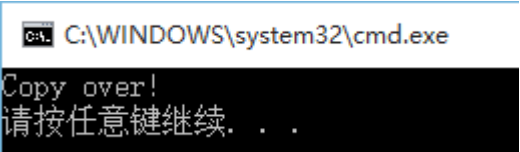
```

其中，指针 buffer 指向内存中一段存储空间。len 是读写的字节数。下面通过一个文件复制程序实例来说明上述函数的使用方法。

实例 5-11 将一个文件中的数据以二进制形式复制到另外一个文件中。

```
#include <iostream>
#include <fstream>
using namespace std;

void main()
{
    //参数为读取的目标文件的路径+文件名
    ifstream fin("G:\\01.mp3", ios::in | ios::binary);
    if (!fin) {
        cout << "File open error!\n";
        return;
    }
    //参数为写入的目标文件的路径+文件名
    ofstream fout("G:\\02.mp3", ios::binary);
    char c[1024];
    while (!fin.eof())
    {
        fin.read(c, 1024);
        fout.write(c, fin.gcount()); //成员函数gcount()是用来计算写的字节数
    }
    fin.close();
    fout.close();
    cout << "Copy over!\n";
}
```



(2) 与文件指针有关的流成员函数

在文件中有一个文件数据读写标记（俗称文件指针），用来指明文件当前进行读写的数据位置。通过文件指针对需要读写的字节进行指向，程序员可以使用表 5-6 中的成员函数对指针进行控制，实现对任意位置的字节进行读写，即所谓随机读写操作。文件指针相关成员函数如表 5-6 所示。

表 5-6 文件指针的控制成员函数

成员函数	作 用
gcount()	返回最后一次输入所读入的字节数
tellg()	返回输入文件指针的当前位置
seekg(文件中的位置)	将输入文件中指针移到指定的位置

seekg(位移量, 参照位置)	以参照位置为基础移动若干字节
tellp()	返回输出文件指针当前的位置
seekp(文件中的位置)	将输出文件中指针移到指定的位置
seekp(位移量, 参照位置)	以参照位置为基础移动若干字节

需要特别说明：

- (1) 这些函数名的第一个字母或最后一个字母不是g（代表get）就是p（代表put）。
- (2) 函数参数中的“文件中的位置”和“位移量”已被指定为long型整数, 以字节为单位。
“参照位置”可以是下面三者之一：

ios::beg文件开头(beg是begin的缩写), 这是默认值。

ios::cur指针当前的位置(cur是current的缩写)。

ios::end文件末尾。

有关使用方法见如下举例。

infile.seekg(800); //文件中的位置指针向文件尾部前移到800字节位置。

infile.seekg(-20, ios::cur); //文件中的位置指针从当前位置后移（往文件头部方向）20字节。

outfile.seekp(-30, ios::end); // 文件中的位置指针从文件尾后移（往文件头部方向）30字节。

下面通过一个程序实例来说明上述成员函数的使用方法。

实例5-12 文件的随机读写操作。

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

void test_write_read_cin_getline()
{
    char data[100];
    ofstream outfile;
    outfile.open("TheMysteryMethod.txt");
    cout << "Writing to the file" << endl;
    cout << "Enter your name: ";
    cin.getline(data, 100);
    outfile << data << endl;
    cout << "Enter your age: ";
    cin >> data;
    cin.ignore();
    outfile << data << endl; //将输入数据再次写入文件
    outfile.close();
    ifstream infile;
    infile.open("TheMysteryMethod.txt"); //以读方式打开文件
    cout << "Reading from the file" << endl;
    infile >> data;
    cout << data << endl; //将数据输出到屏幕
```

```

    infile >> data;    //从文件中再次读取数据并输出到屏幕
    cout << data << endl;
    infile.close();
}
void test_write()
{
    ofstream myfile;
    myfile.open("TheMysteryMethod.txt", ios::app);
    if (myfile.is_open())
    {
        myfile << "\nTo recap, the three main objectives in the Mystery Method are: \n"
                "To attract a woman \n"
                "To establish comfort, trust, and connection \n"
                "To structure the opportunity to be seduced \n";
        myfile.close();
    }
    else
        cout << "打开文件失败! \n";
}
void test_write_read_getline()
{
    string str;
    ofstream a_file("TheMysteryMethod.txt", ios::app); //追加方式
    if (a_file.is_open())
    {
        a_file << "A woman's number-one emotional priority is safety and security.";
        a_file.close();
    }
    else
        cout << "Unable to open file\n";
    ifstream b_file("TheMysteryMethod.txt", ios::app); //打开读文件，追加方式
    b_file >> str; //只显示to 表示遇到空格停止接收字符
    cout << str << "\n";
    getline(b_file, str, '\0'); //输出缓冲区剩余的字符
    cout << str << "\n";
    cin.get(); // wait for a keypress
}
void GetSizeOfFile(string filename)
{
    long begin, end;
    ifstream myfile(filename.c_str()); //必须转换为c风格字符串
    begin = myfile.tellg();
    myfile.seekg(0, ios::end);
    end = myfile.tellg();
}

```

```

myfile.close();
cout << "File Size : " << end - begin << endl;
}
int main()
{
    test_write_read_cin_getline();
    test_write();
    test_write_read_getline();
    string file("TheMysteryMethod.txt");
    GetSizeOfFile(file);
    return 0;
}

```

程序的运行结果为:

```

C:\WINDOWS\system32\cmd.exe
Writing to the file
Enter your name: Tao Huaizhou
Enter your age: 26
Reading from the file
Tao
Huaizhou
Tao
Huaizhou
26

To recap, the three main objectives in the Mystery Method are:
To attract a woman
To establish comfort, trust, and connection
To structure the opportunity to be seduced
A woman's number-one emotional priority is safety and security.

File Size : 260
请按任意键继续. . .

```

```

Writing to the file
Enter your name: Tao Huaizhou
Enter your age: 26
Reading from the file
Tao
Huaizhou
Tao
Huaizhou
26

```


To recap, the three main objectives in the Mystery Method are:

To attract a woman

To establish comfort, trust, and connection

To structure the opportunity to be seduced

A woman's number-one emotional priority is safety and security.

File Size : 260

请按任意键继续. . .

5. 4 字符串流的输入和输出

在实际应用中, 为了进一步提高程序的输入和输出速度, 可以使用 C++ 的字符串流机制。字符串流是以内存中用户定义的字符数组(字符串)为输入输出的对象。也就是说, 在应用程序开发时, 可以设立一个字符数组作为临时存储空间, 程序的数据可以输出到这个字符数组中, 或者从字符数组(字符串)将数据读入程序中。因此, 有时将这种输入和输出方式也称为内存流。

尽管这个内存的临时空间是以字符数组方式设立。但实际上可以存放字符、整数、浮点数以及其他类型的数据。但在向字符数组存入不同类型的数据之前, 需要先将这些数据从二进制形式转换为 ASCII 代码, 然后存放到字符数组中。从字符数组读数据程序的对象(变量)中时, 先将字符数组中的数据从 ASCII 代码转换为二进制形式, 然后再读入到程序中的不同类型的变量中。因此, 在使用字符串流进行输入/输出时, 系统完成相应数据类型到 ASCII 代码格式的转换。

5. 4. 1 字符串流和字符串流对象

C++ 在 Strstream 头文件中, 声明了 3 个字符串流类 `istrstream`、`ostrstream` 和 `strstream`。而且文件流类和字符串流类都是 `ostream`、`istream` 和 `iostream` 类的派生类。因此, 向内存中的字符数组写数据就如同向文件写数据方式。但也存在一些差异。例如, 字符串流对象关联的不是文件, 而是内存中的一个字符数组, 字符串流对象的读写速度比文件流对象要快得多。而且, 文件需要打开和关闭, 但字符串流只需要构建字符串流对象, 不存在打开和关闭的问题。另外, 文件的最后都有一个文件结束符。而字符串流所关联的字符数组中没有相应的结束标志。因此, 用户在向字符数组写入全部数据后, 还需要写入一个特殊字符作为结束符。

字符串流对象的构建方法有 3 种。

1. 输出字符串流对象的构建

在建立字符串流对象时, 通过调用一个有参构造函数, 给定一些实际参数来确立字符串流与字符数组的关联。建立字符串流对象的方法与含义如下。

`ostrstream` 类构造函数原型为 `ostrstream::ostrstream(char *buffer, int n, int mode=ios::out);`

其中, `buffer` 是指向字符数组首元素的指针, `n` 为指定的流缓冲区的大小; 第 3 个参数是可选的, 默认为 `ios::out` 方式。

例如, `char ch1[200];` // 定义长度为 200 字节的字符数据, 作为字符串流的缓冲区

`ostream stroutput(ch1, 100);` // 建立输出字符串流对象 `stroutput`, 并使 `stroutput` 与字符数组 `ch1` 关联, 流缓冲区大小为 100。

2. 输入字符串流对象的构建

`istream` 类提供了两个带参的构造函数, 原型分别如下。

`istream::istream(char *buffer);`

`istream::istream(char *buffer, int n);`

其中, `buffer` 是指向字符数组首元素的指针, 用它来初始化流对象(使流对象与字符数组建立关联)。

例如, `char ch2[200];` // 定义长度为 200 的字符数组, 作为输入字符串流的缓冲区

`istream strinput(ch2);` // 建立输入字符串流对象 `strinput`, 将字符数组 `ch2` 中的全部数据作为输入字符串流的内容。

或者, `istream strinput(ch2, 100);` // 建立输入字符串流对象 `strinput`, 流缓冲区大小为 100。

3. 输入输出字符串流对象的构建

`stringstream` 类提供的构造函数的原型为 `stringstream::stringstream(char *buffer, int n, int mode);`

其中, `buffer` 是指向字符数组首元素的指针, 使流对象与字符数组建立关联。

例如, `char ch3[100];` // 定义长度为 100 的字符数组, 作为输入输出字符串流的缓冲区

`stringstream strio(ch3, sizeof(ch3), ios::in|ios::out);` // 建立输入输出字符串流对象, 以字符数组 `ch3` 为输入输出对象

C++ 中的字符串流对象主要用处在对多种数据类型的转换, 下面结合程序实例来说明字符串流及其流对象的使用方法。

实例 5-13 字符串流对象在数据类型转换中应用

```
#include <sstream>
#include <iostream>
#include <string>
using namespace std;
int main(void)
{
    string s = "carea 89 男";
    istream sin(s.c_str());
    string name;
    int age;
    string sex;
    sin >> name >> age >> sex;
```

```
cout << "姓名: " << name << endl
    << "年龄: " << age << endl
    << "性别: " << sex << endl;
return 0;
}
```

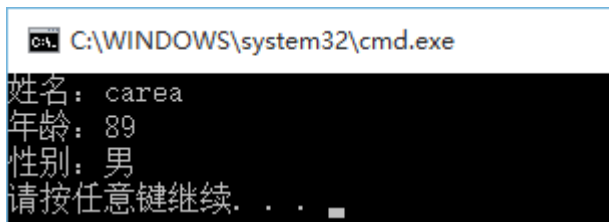
运行结果:

姓名: carea

年龄: 89

性别: 男

如图所示。



为了进一步了解上述字符串流类及其对象的构建方法,下面通过一个完整应用程序来说其使用中的应注意问题。

实例 5-14 字符串流类的应用

```
#include<strstream>
#include<iostream>
#include<cstring>
using namespace std;
int main() {
    int i;
    char str[36] = "This is a book.";
    char ch;
    istrstream input(str, 36);    //以串流为信息源
    ostrstream output(str, 36);
    cout << "字符串长度: " << strlen(str) << endl;
    for (i = 0; i<36; i++)
    {
        input >> ch;    //从输入设备(串)读入一个字符,所有空白字符全跳过
        cout << ch;    //输出字符
    }
    cout << endl;
    int inum1 = 93, inum2;
    double fnum1 = 89.5, fnum2;
    output << inum1 << ' ' << fnum1 << '\0';    //加空格分隔数字
    cout << "字符串长度: " << strlen(str) << endl;
    istrstream input1(str, 0);    //第二参数为0时,表示连接到以串结束符终结的串
    input1 >> inum2 >> fnum2;
```

```

cout << "整数: " << inum2 << '\t' << "浮点数: " << fnum2 << endl; //输出: 整数: 93 浮点数:
89.5
cout << "字符串长度: " << strlen(str) << endl;
return 0;
}

```

运行结果:

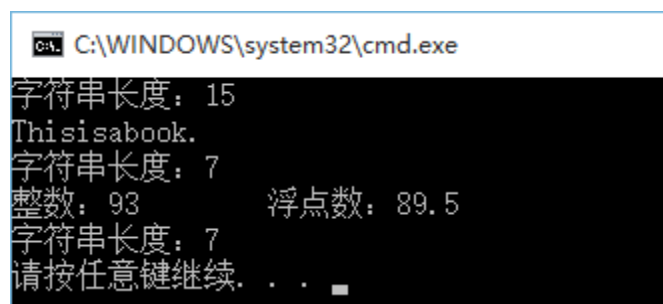
字符串长度: 15

Thisisabook.

字符串长度: 7

整数: 93 浮点数: 89.5

字符串长度: 7



通过分析上述程序, 在应用字符串流对象时, 应该主要注意如下问题:

- (1) 字符数组中的数据全部是以 ASCII 代码形式存放, 而不是以二进制形式表示的数据。
- (2) 一般都把流缓冲区的大小指定与字符数组的大小相同。
- (3) 通过字符串流从字符数组读数据就如同从键盘读数据一样, 可以从字符串流读入字符数据, 也可以读入整数、浮点数或其他类型数据。
- (4) 字符串流关联的字符数组并不一定是专为字符串流而定义的数组, 它与一般的字符数组无异, 可以对该数组进行其他各种操作。

字符流机制的核心思想: 与字符串流关联的字符数组相当于内存中的临时 ASCII 文件, 可以以 ASCII 形式存放各种类型的数据。对它读写数据的用法相当于标准设备 (显示器与键盘), 但标准设备不能保存数据, 字符串流可以暂时保存数据。另外, 字符串流比磁盘文件使用方便, 不必建立文件 (不需打开与关闭), 存取速度快。其缺点是生命周期与其所在的模块 (如主函数) 相同, 程序结束了, 字符数组也不存在了, 保持其中的数据也就没有了。

5.5 综合程序应用—公司人事管理系统

在前面一章的综合程序应用的基础上, 对 “公司人事管理系统” 进行功能扩展和修改。要求如下:

- (1) 给 “XX 公司人事管理系统” 设计一个主界面, 采用 C++I/O 操作在屏幕打印一个漂亮的图案和菜单。最少实现 4 个菜单的功能: 存盘、修改、查询和退出;

- (2) 存盘功能：对新职工输入的数据能保存在一个文件后面；
- (3) 修改功能：按照工号读出职工信息，修改后写入原来位置；
- (4) 查询功能：输入一个姓名或工号，能从文件中读出该职工的有关信息；
- (5) 退出功能：正常退出程序，道声“BYEBYE”。

由于篇幅限定。在此不再列举程序代码。程序代码可以参考后面一章的程序源码中的相关内容。

本章小结

本章主要介绍 C++ 输出和输入操作中核心概念“流”，以及 C++ 定义的相关类库。重点介绍了文件操作，标准输出/输入操作和字符串流操作等方法。在文件操作中，需要区分文本文件和二进制（内存映像）文件的差别。同时，熟悉文件操作“三部曲”：即文件打开，文件读写和文件关闭。而且需要了解对文本文件和二进制文件进行读写时差别以及相关的成员函数。在标柱设备的输入/输出操作时，需要了解系统定义的 4 个全局对象含义，即 `cout`、`cin`、`cerr` 和 `clog`。另外在学习字符串流时，一定需要懂得引入该机制目的和用途，以及与文件、标准设备输出输入的差异。

课后习题

1. 为什么 `cin` 输入时，空格和回车无法读入？这时可改用哪些流成员函数？
2. 简述文本文件和二进制文件在存储格式、读写方式等方面的不同，各自的优点和缺点。
3. 文本文件可以按行也可以按字符进行复制，在使用中为保证能完整复制要注意哪些问题？
4. 文件的随机访问为什么总是用二进制文件，而不用文本文件？怎样使用 `istream` 和 `ostream` 的成员函数来实现随机访问文件？
5. 建立一个文本文件，从键盘输入一篇短文存放在该文件中短文由若干行构成，每行不超过 30 个字符。然后将文本文件读出，显示在屏幕上并统计该文件的行数。
6. 建立某单位职工通信录二进制文件，文件中的每个记录包括职工编号、姓名、电话号码、邮政编码和住址。从键盘上输入职工的编号，在由所建立的通信录文件中查找该职工资料。查找成功时，显示职工的姓名、电话号码、邮政编码和住址。
7. 设有两个按升序排列的二进制数据文件 `f` 和 `g`，将它们合并生成一个新的升序二进制数据文件 `h`。
8. 在第 5 题基础上，采用字符串流类的相关概念，修改程序，将键盘读入的数据先存放在数组 `c` 中，然后在写入到文件。接着从文件中再读入到数组 `d` 中，在从数组 `d` 输出到屏幕。
9. 阅读下列程序，写出执行结果

```
#include<iostream>
using namespace std;
void main()
```

```
{  
    double x=123.456;  
    cout.width(10);  
    cout.setf(ios::dec, ios::basefield);  
    cout<<x<<endl;  
    cout.setf(ios::left);  
    cout<<x<<endl;  
    cout.width(15);  
    cout.setf(ios::right, ios::left);  
    cout<<x<<endl;  
    cout.setf(ios::showpos);  
    cout<<x<<endl;  
    cout<<-x<<endl;  
    cout.setf(ios::scientific);  
    cout<<x<<endl;  
}
```